

AI introduction

Systemudvikling

Plan:

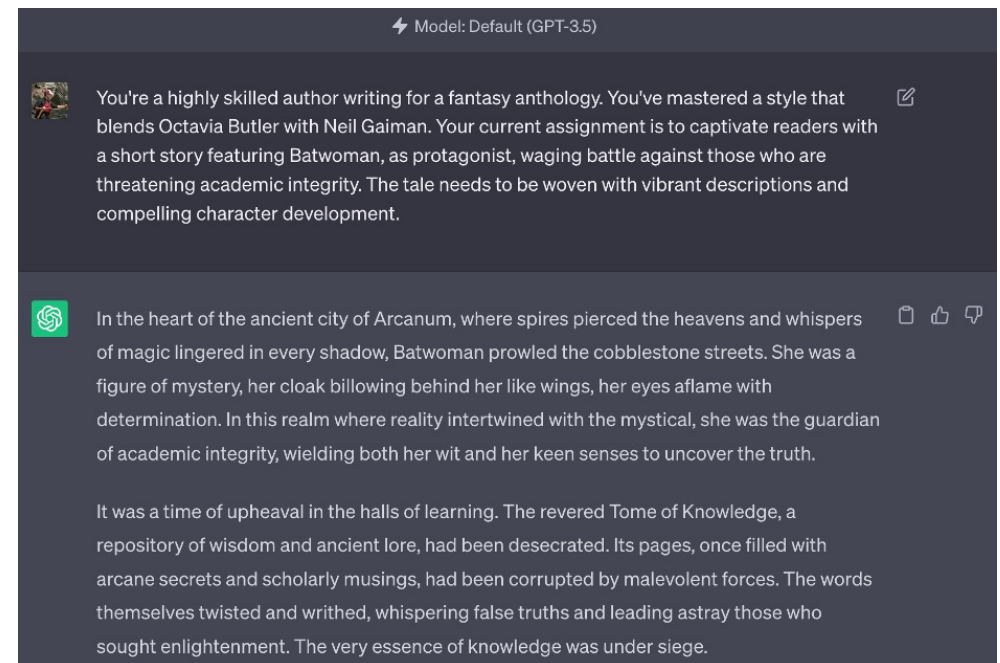
- Module I (29/9):
 - Session I: What is AI? What are AI agents?
 - Session II: Coding agents and spec driven development
 - Session III: Case – Constitution, specify and clarify with AI
- Module II (6/10):
 - Plan and task part of spec driven development
 - Code clickable POC with coding agent (case)
 - Evaluate POC (case)

LLM og chat bots

- Vi kender ChatGTP, Gemini, Claude, Grok, etc.
- Menneske som spørger en chatbot

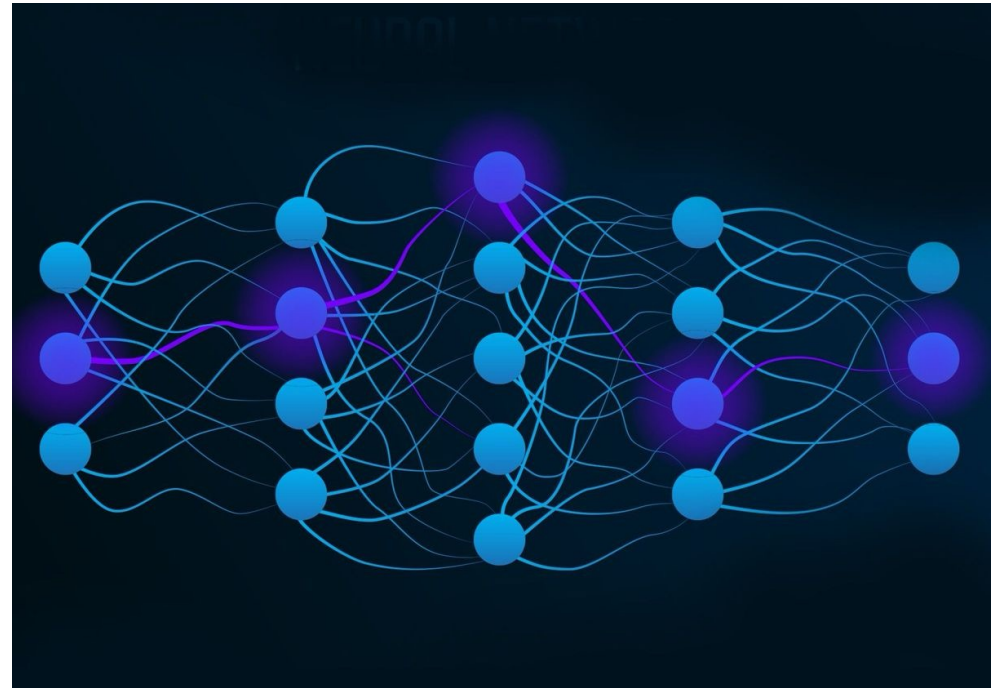
What is an LLM?

- Svarer på naturligt sprog udfra det den har lært
- En stor sandsynligheds maskine (det kommer vi tilbage til)



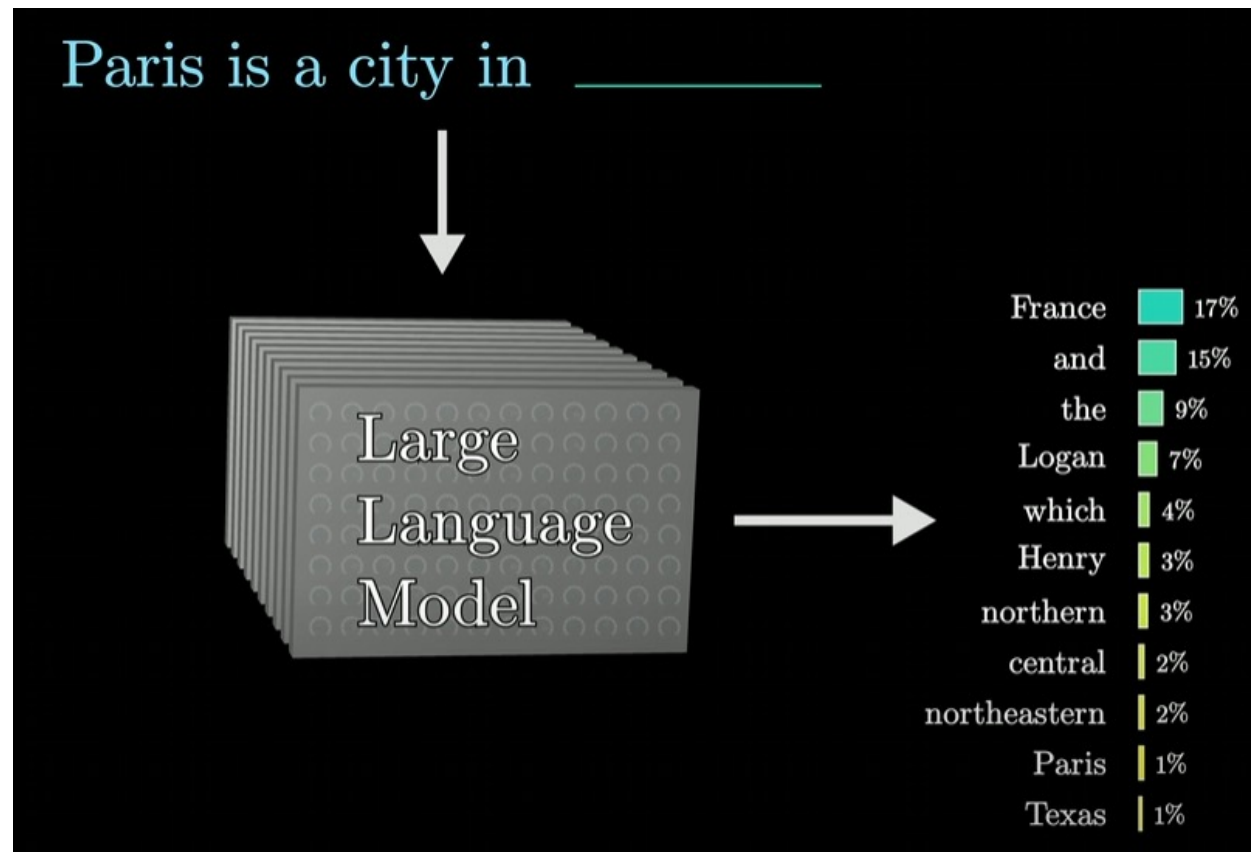
Neurale netværk

- Billions of parameters
 - Inputs
 - Weights
 - Etc.



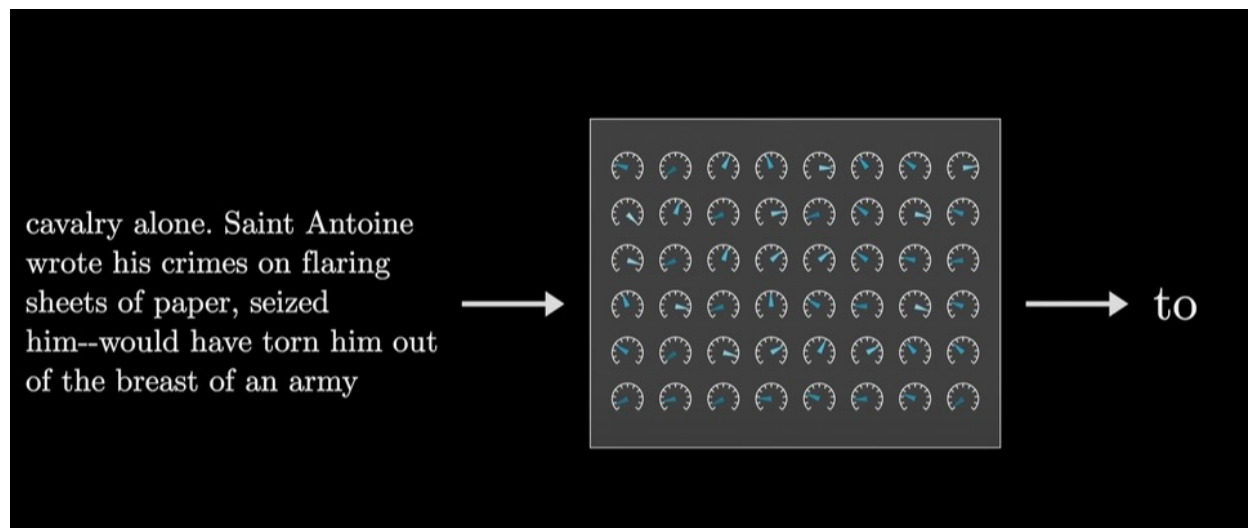
LLMs guesses the next word

- A LLM is trained to guess the next most likely word
- It is making qualified "guesses"
- Sometimes it guesses "wrong"



Træningsdata

- Text fra internettet
- Bruger trilliarder af eksempler
- Tager måneder at træne de største modeller



Tokenization and the Token Window

- LLMs don't use words but break down text into smaller units called tokens
- Tokens are whole words or a part of a word
 - "Amoral" might be 2 tokens "a" and "moral"
- The context window is the maximum number of tokens an LLM can "remember" or consider at one time when generating a response
 - "It is the input layer to the neural network"
 - A 100K context windows can approximately contain 75k words

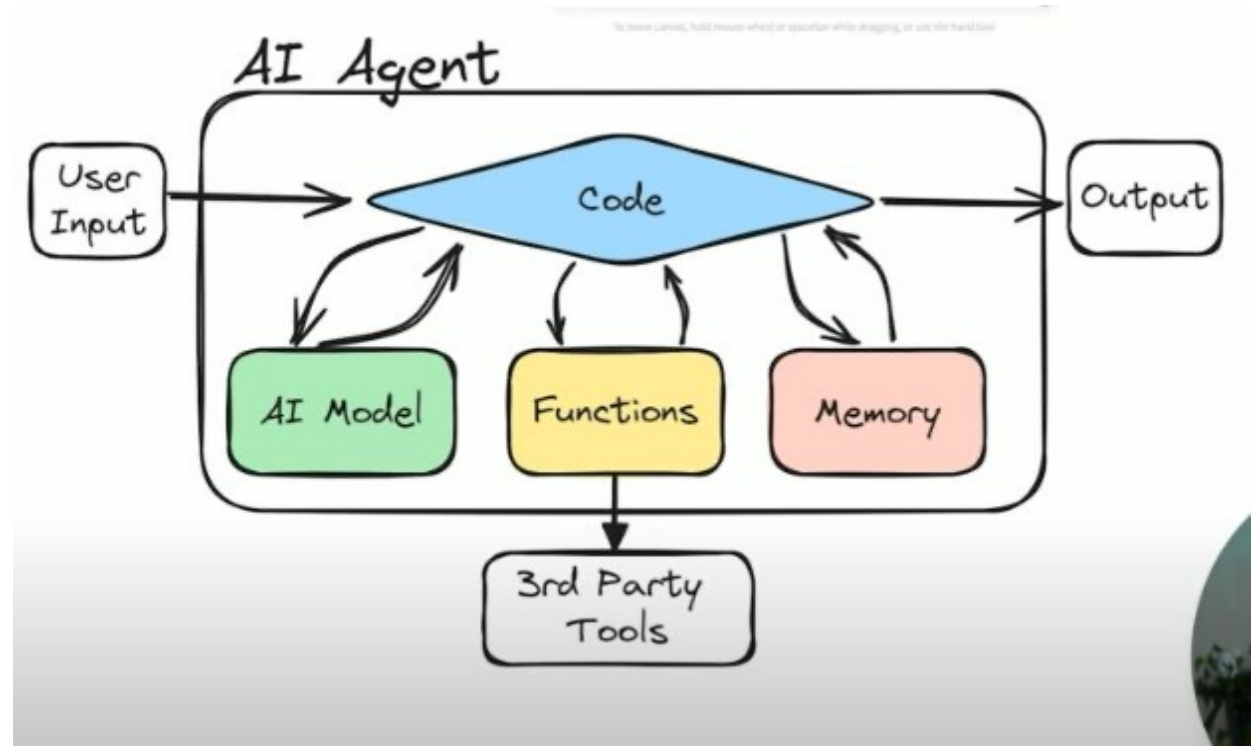
LLM limitations

- **Hallucinations:** LLMs can generate plausible but incorrect or nonsensical information.
- **Bias:** They can reflect and amplify biases present in their training data.
- **Lack of True Understanding:** At their core, LLMs are probabilistic models predicting the next token; they do not possess true consciousness or understanding in the human sense.
- **Computational Cost:** Training and deploying large-scale LLMs requires significant computational resources.

AI Agenter

AI agenter

- Vi sætter et stykke software foran LLMen – også kendt som en “harness”
- Vi kan "fodre" LLM med data, som den kan reagere på



AI agents

An Agent is a system that leverages an AI model to interact with its environment in order to achieve a user-defined objective.

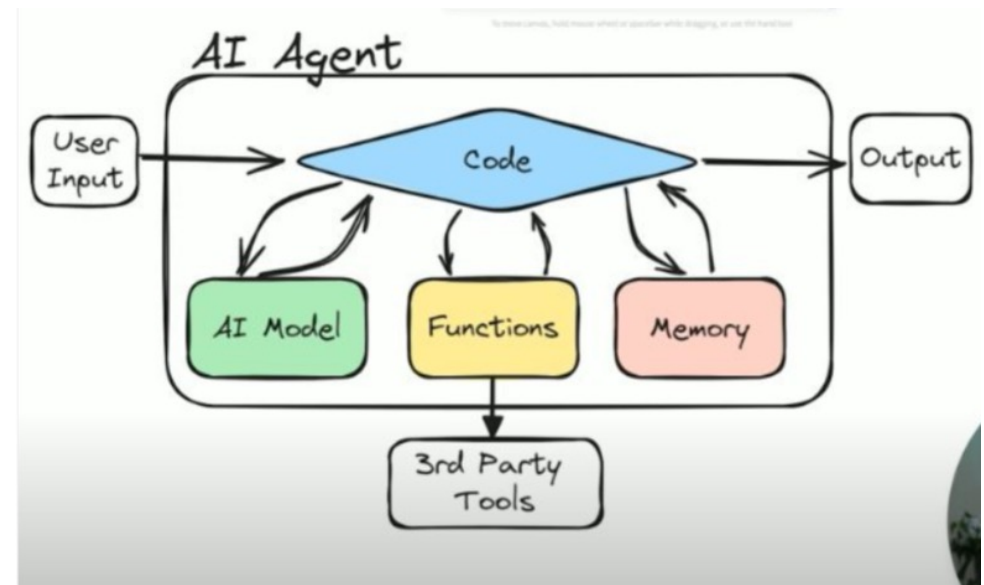
It combines reasoning, planning, and the execution of actions (often via external tools) to fulfill tasks.



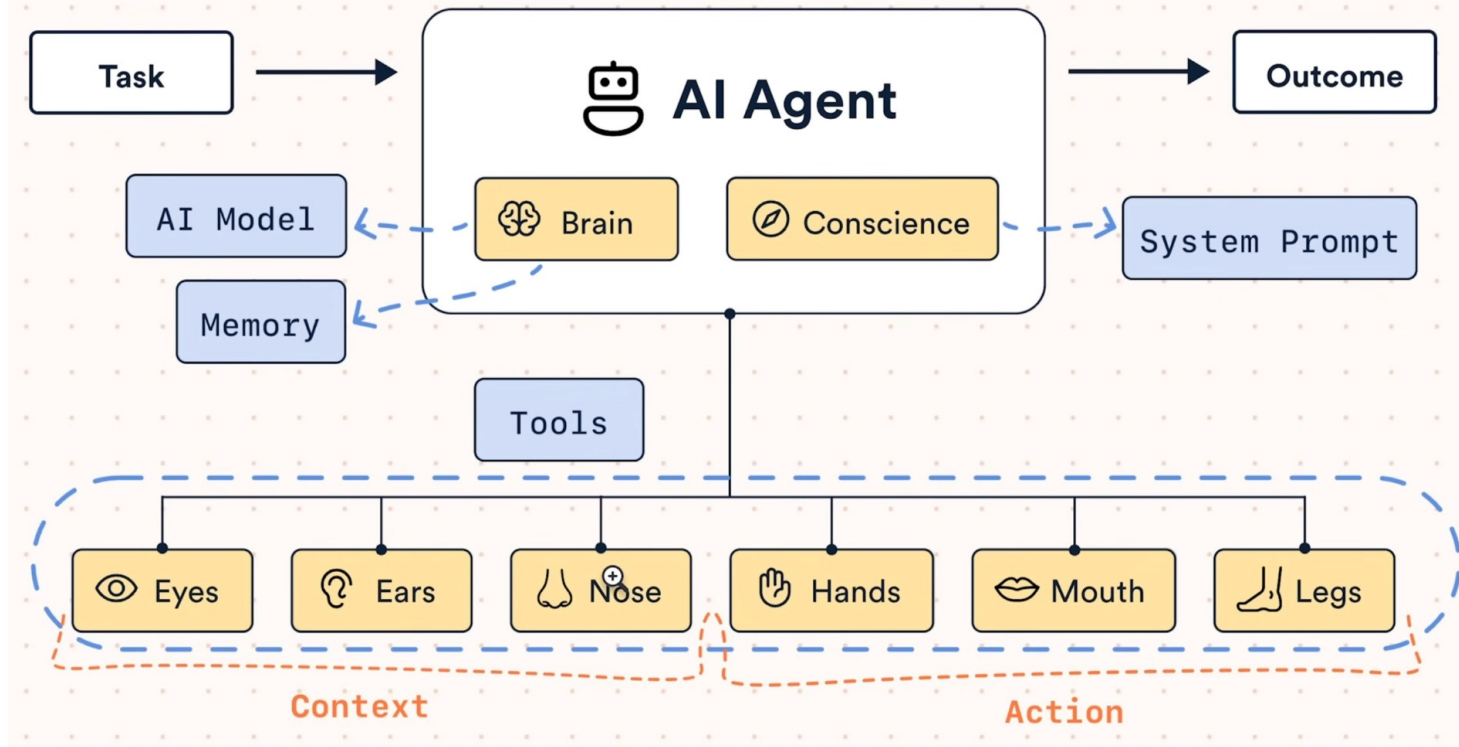
Hvad er en AI-agent?

Nøglefunktioner:

- **Ræsonnement:** Planlægger trin mod et mål.
- **Hukommelse:** Husker tidligere kontekst.
- **Værktøjer:** Interagerer med API'er/databaser.



Imagine an AI agent is like a person



Imagine an AI agent is like a person – Brain, Conscience, AI Model, Memory, System Prompt, Tools (Eyes, Ears, Nose, Hands, Mouth, Legs)

LLM vs. AI Agent

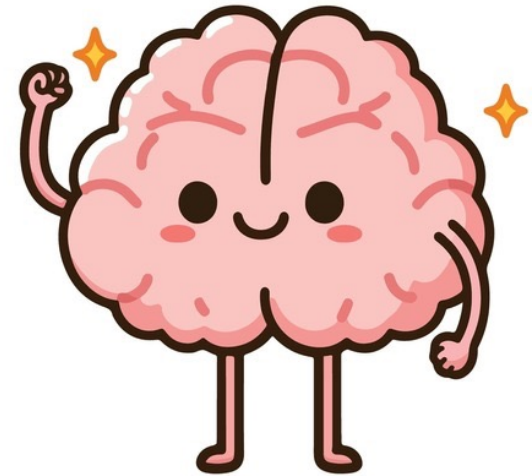
Feature	LLM	AI Agent
Core Capability	Text generation	Goal-oriented task completion
Decision-Making	None	Yes
Uses Tools/APIs	No	Yes
Workflow Complexity	Single-step	Multi-step
Scope	Generates language	Performs complex, real-world tasks
Example	LLM generating a paragraph	An agent scheduling an appointment

Elements of an AI Agent

- Perception
 - It's the ability to gather information from its environment.
- Decision-making
 - This is the agent's "brain." Based on its perception (the information gathered) and its programmed goals, the agent decides what to do next.
 - LLMs, rule-based systems, Machine learning models (classifications)
 - The agent can plan ahead ("think")
- Action
 - Once a decision is made, the agent needs to act upon it.
 - Send message to user, call an API, run a workflow, update DB, etc.
 - Happens through definition of tools or function calling
- Memory
 - Agents often need to remember past interactions or learned information to provide context for future decisions.
 - Recall previous interaction
 - Knowledge databases (RAGs)

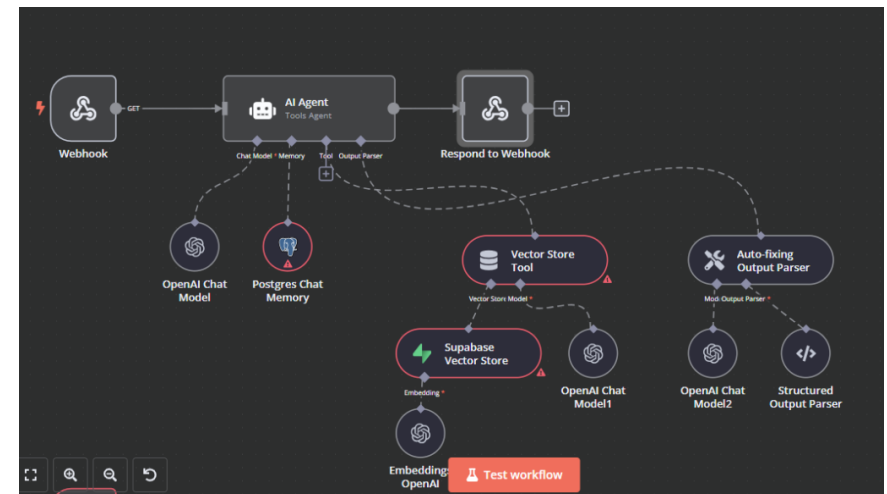
The brain (AI models)

- This is where all the thinking happens.
 - The AI model handles reasoning and planning.
 - It decides which Actions to take based on the situation.
-
- The brain is your LLM



The Body (Capabilities and Tools)

- This part represents everything the Agent is equipped to do.
- The scope of possible actions depends on what the agent has been equipped with.
 - For example, because humans lack wings, they can't perform the “fly” Action, but they can execute Actions like “walk”, “run”, “jump”, “grab”, and so on.



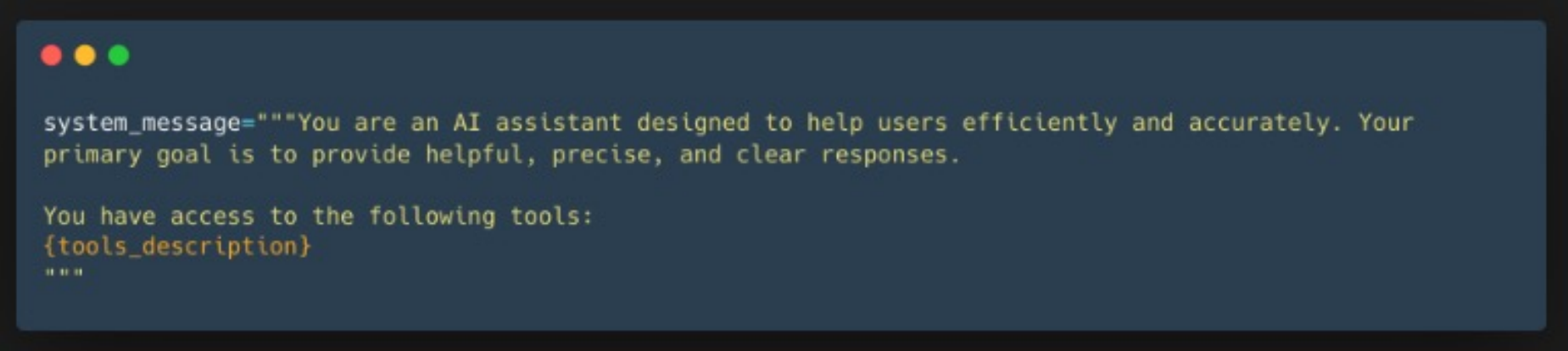
Tools should...

- A good tool should be something that **complements the power of an LLM**.
 - For instance, if you need to perform arithmetic, giving a **calculator tool** to your LLM will provide better results than relying on the native capabilities of the model.
 - **LLMs predict the completion of a prompt based on their training data**, which means that their internal knowledge only includes events prior to their training. Therefore, if your agent needs up-to-date data you must provide it through some tool.
- A Tool should contain:
 - A **textual description of what the function does**.
 - A *Callable* (something to perform an action).
 - *Arguments* with typings.
 - (Optional) Outputs with typings.



Tools to the agent

- For this to work, we must be very precise and accurate about:
 - What the tool does
 - What exact inputs it expects



```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
{tools_description}
"""
```

Calculator tool

- Tool Name: calculator,
- Description: Multiply two integers.,
- Arguments: a: int, b: int,
- Outputs: int

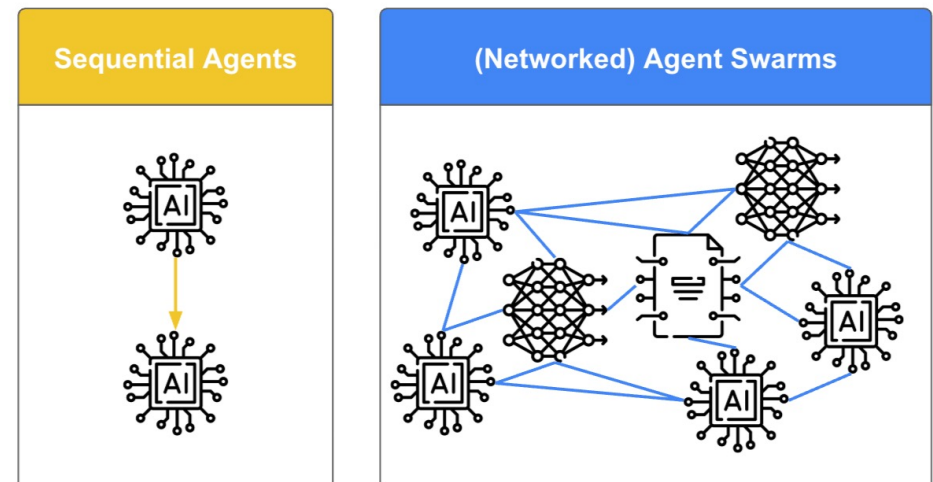
Spectrum of agents

- Agents exist on a continuous spectrum of increasing agency

Agency Level	Description	What that's called	Example pattern
☆☆☆	Agent output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
☆☆☆	Agent output determines basic control flow	Router	<code>if llm_decision(): path_a() else: path_b()</code>
☆☆☆	Agent output determines function execution	Tool caller	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
☆☆☆	Agent output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
☆☆☆	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>

Architecture – sequential agents

- Chain the AI agents
- One agent's output feeds into the next, creating a linear workflow
- Parallel agents execute multiple tasks simultaneously
- Serial execution is suitable for predictable, multi-step processes
- Parallel execution is better for complex problems requiring simultaneous effort and faster overall completion, enabling more efficient resource use and breaking down large tasks.



Prompt Engineering

How do we instruct our agents?

Definition – what is prompt engineering

- Prompt engineering is the practice of carefully designing, crafting, and refining the input text (prompts) given to generative AI models to elicit the most accurate, relevant, and desired output.

A structure for AI Agent prompts

- Roles
- Tasks
- Requirements
- Instructions

Role

- Defines the persona the model should adopt. Influences tone, knowledge, and style.
- "Act as a senior software architect."

Task

- The specific, clear objective for the model.
- "Outline a project plan for a new microservices application."

Requirements

- Conditions that dictate the output's format and constraints.
- "The outline should be a numbered list with a maximum of five top-level items."

Instructions

- Guidance on how to execute the task. Provides intermediate steps or examples.
- "For each microservice, provide a brief description and a list of its core dependencies."

Hvad er Markdown?

- Et simpelt formateringsprog baseret på tegnsætning (fx #, *, -)
- Det er "plain text" - stærkt forsimplet udgave af html
- Markdown bruges af Aler og ChatGPT til at gøre tekst overskuelig, læsbar og organiseret

Mayer

For the long-form writer, established blogger, and inspired author.

Features

- * Responsive / Mobile Ready
- * Internationalized
- * Social Networks (Facebook, Pinterest, Email, Twitter, Google+)
- * Readability
- * Editor Styles
- * Personalized For The Author
- * Theme Customizer / No Options
- * Navigation / Simplicity / Less for the user to consider
- * Write. Install / Write

Testimonials

- * TODO

Pricing

- * TODO

Pages

- * The Blog
- * About (Pressware / Tom McFarlin)
- * Documentation / Manual

Hvorfor Markdown

- Let læseligt for Aler
- Læseligt for mennesker
- Alle værktøjer kan læse det
- Renderer fint alle steder
- Bliver ikke korrumpet af forskellige versioner af software.
- "Skriv en gang, brug alle steder"

Strukturering af markdown

Guide: Kom godt i gang med dit projekt

1. Definér *dine* mål

Start med at sætte klare, målbare mål for projektet.

2. Identificér interessenter

Overvej:

- Hvem bliver **påvirket**?
- Hvem skal **involveres**?

3. Læg en plan

Inddrag:

- Tidslinje
- Ressourcer
- Ansvarsområder

4. Sæt deadlines og følg op

Det er **vigtigt** at:

- Fastsætte *realistiske* deadlines
- Løbende evaluere fremdriften



Guide: Kom godt i gang med dit projekt

1. Definér *dine* mål

Start med at sætte klare, målbare mål for projektet.

2. Identificér interessenter

Overvej:

- Hvem bliver **påvirket**?
- Hvem skal **involveres**?

3. Læg en plan

Inddrag:

- Tidslinje
- Ressourcer
- Ansvarsområder

4. Sæt deadlines og følg op

Det er **vigtigt** at:

- Fastsætte *realistiske* deadlines
- Løbende evaluere fremdriften

Strukturering med Markdown

Formål	Markdown-syntaks	Resultat
Overskrift 1	# Overskrift	Overskrift
Overskrift 2	## Overskrift	Overskrift
Overskrift 3	### Overskrift	Overskrift
Overskrift 4	#### Overskrift	Overskrift
Fed tekst	**fed tekst**	fed tekst
Kursiv tekst	<i>*kursiv tekst*</i>	<i>kursiv tekst</i>
Fed + kursiv	***fed og kursiv***	<i>fed og kursiv</i>
Punktopstilling	- Punkt 1 - Punkt 2	• Punkt 1 • Punkt 2
Nummereret liste	1. Første 2. Anden	1. Første 2. Anden

Coding agents

From chatbots to coding agents

- Codex and Claude Code are coding agents: AI tools designed to work with software projects
- They use a large language model (LLM) to understand requests, reason about possible steps, and produce text or code
- Unlike a basic chatbot, a coding agent can also use tools to work with the project
- You give it a goal; it can inspect the project and propose or carry out changes

What can a coding agent do?

- Read and search project files to understand how the software is organized
- Create, edit, and sometimes delete files, depending on its permissions
- Run commands such as tests or the application, when permitted
- Use other tools—such as web search or Git—when those tools are available and enabled
- Show its changes so a person can review them

The agent is more than the LLM

- **LLM:** interprets the request and generates possible next steps and outputs.
- **Tools:** let the agent inspect files, edit code, run commands, or access other services.
- **Instructions and context:** help it follow project conventions and understand the task.
- **Permissions and approvals:** limit what it can do and when it needs a person to approve an action.

Together, the model, tools, context, and control loop make it a coding agent. It can work on files and run checks—not just suggest code in a chat window.

We'll use a coding agent for requirements

- A coding agent can read the case materials and help the group draft requirements and specifications
- It can ask questions, identify gaps, and revise Markdown documents as the group works
- It can also help create a plan, break work into tasks, and later write and test code
- The group supplies the case knowledge, checks the agent's claims, and decides what to accept

The agent helps with the work; the group owns the requirements and the decisions.

Skills



Hvad er en skill?

En **skill** er et dokument agenten kan slå op i, når den har brug for det.

Uden skills

Agenten ved kun hvad den har lært under træning. Ny opgave = ny prompt hver gang.

Med skills

Agenten finder selv den relevante skill og følger instruktionerne — præcis og konsistent.

Tænk på det som en **Standard Operating Procedure** agenten selv kan læse og følge.

Skills: Hvordan virker det?

Agenten holder en liste over alle tilgængelige skills (navn + beskrivelse). Når du giver en opgave, matcher den opgaven mod listen og indlæser den relevante skill.

Bruger: "Giv mig dagens AI-nyheder"

↓

Agent finder: ai-researcher (matches "AI-nyheder")

↓

Indlæser SKILL.md og følger instruktionerne

↓

Kører fetch-script → formaterer → sender til Telegram

Skills er **token-effektive** — kun beskrivelsen er altid indlæst. Selve instruktionerne hentes kun når de bruges.

MD files

- AGENTS.md
 - instructions for how the coding agent should work in this repository
- SOUL.md
 - the agent's character, values, and communication style
- Memory.md
 - useful project context and decisions to carry across sessions
- README.md
 - What the project is all about
- CLAUDE.md
 - Claude Code's project instructions and memory file
- SKILL.md
 - reusable instructions for a specific task or workflow, supported through each tool's skill mechanism

Spec Driven Development

Github Spec Kit

What is Spec-Driven Development?

- First describe the problem and what the software should do
- Turn that description into clear requirements and ways to check success
- Use those requirements to guide the technical plan, tasks, and implementation
- Keep checking that the implementation still matches the specification

Why use it with AI?

- AI coding agents can make assumptions when a request is vague
- A specification gives the agent shared context, constraints, and a clear goal
- This can reduce misunderstandings, unsupported assumptions, and drift from the intended feature
- It does not prevent every mistake: people must review the requirements, plan, and code

GitHub Spec Kit: the helicopter view

- GitHub Spec Kit is an open-source toolkit for working with a coding agent using a specification-first process
 - The team starts with an idea and develops project principles, a feature specification, a technical plan, and implementation tasks
 - The coding agent helps create and work through these artifacts
 - The team reviews the output and makes the decisions; Spec Kit does not replace teamwork or judgment
-
- <https://github.com/github/spec-kit>

The workflow

Constitution → Specify → Clarify → Plan → Tasks → Implement → Converge

- **Constitution:** agree on project-wide principles and constraints.
- **Specify:** describe the feature, users, requirements, and success criteria.
- **Clarify:** find and resolve gaps, ambiguity, and unanswered questions.
- **Plan:** choose the technical approach and design.
- **Tasks:** break the work into implementable steps.
- **Implement:** build and test the selected work.
- **Converge:** check what is complete and identify remaining gaps.

Checklists and Analyze provide extra quality and consistency checks along the way.

What does Spec Kit provide?

- **Agent instructions (skills/commands):** guide the coding agent through each step
- **Templates:** give the team a consistent structure for specifications, plans, and tasks.
- **A repeatable workflow:** makes it easier to see what has been decided and what comes next.
- **Quality checks:** help find missing requirements or inconsistencies between artifacts.

The agent proposes and produces; the group reviews, corrects, and approves.

Case:
Bøgegårdens Ridecenter

Start specifying with AI

- Give the coding agent your case findings, interview notes, and relevant constraints
- Ask it to draft a **Constitution** for the project
- Review and correct the Constitution as a group
- Use **Specify** to draft requirements for the whole case
- Use **Clarify** to identify assumptions, conflicts, and unanswered questions
- Review the resulting specification together

Ready for Module 2

Before Module 2, finish reviewing the specification. It should include:

- The important users, goals, and workflows
- Requirements and success criteria that can be checked
- Assumptions and unanswered questions clearly marked
- Your group's corrections and decisions—not unreviewed AI output.

In Module 2, you'll choose **one feature** from the specification to plan, build as a clickable POC, and evaluate.

Working with your AI agent

- **Slow it down:** Ask it to explain its work so you can review each step
- **Stay in control:** Decide whether your group drafts the user stories or asks the AI for a first draft—then review and revise them together
- **Check where you are:** Ask the AI which Spec Kit phase you're in and what you should review next
- **Get another perspective:** Ask a second agent or session to review your specification

Next module: use your specification to plan and build a simple, clickable POC

More tips from the AI...

- One task at a time: Ask for a draft, review it, then move to the next Spec Kit step.
- Ask for evidence: For each requirement, ask which case fact or interview finding supports it. Mark unsupported items as assumptions or open questions
- Review changes before accepting them: Ask which files were changed and why; inspect the changes, especially after broad edits.
- Don't let it invent decisions: Ask the agent to pause and ask your group when a business rule or priority is missing.
- Test its claims: Check the specification and prototype against the acceptance criteria, including realistic exceptions.
- Protect case data: Use anonymised or synthetic personal data in shared repositories and prototypes.